

CUDA Benchmarking Best Practices

Phase 1: Stabilizing System Performance

- Stabilize CPU/GPU
 - Why: OS will scale CPU freq. to save power or increase power on demand. Leads to fluctuations
 - How: CPU governor set to performance to avoid underclock
 - Disable Turbo Boost to prevent thermal throttle
 - Shown to reduce deviation of launch latency
 - Warmup kernels to stabilize CPU/GPU driver handshake
 - Loads CUDA context (mem pools, buffers, channels)
 - Ramps GPU/mem clocks to speed, warms cache
 - Puts chip into thermal/perf steady-state
 - Library busy init, picks algos, loads environ
- NUMA (Non uniform memory access)
 - What: Allocates memory node closest to the target GPU
 - Why: Access from non-local nodes results in higher latency and lower bandwidth
 - Note: Mainly affects systems with multiple GPUs and CPUs.
 - How: Identify NUMA nodes closest to target GPU and force CPU to run on that node
 - Tool: numactl ← look into later, not needed right now
- GPU Clock Settings
 - What: Locks GPU clocks to default application values.
 - Why: GPU clocks can fluctuate based on workload's power draw and thermal limits
 - Ensure every run is at same speed
 - How: Enable Persistent Mode
 - Make settings stick
 - Identify Default Application Clocks
 - `nvidia-smi -q -d SUPPORTED_CLOCKS`
 - Lock Clocks
 - `nvidia-smi lgc <Default Graphics Clock>`
- Note: Not applicable for WSL/Windows

Phase 2: Measuring the Right Thing

- Prevent JIT (Just-In-Time) Compilation

- Why: If binary doesn't support target architecture, driver has to compile PTX during runtime

like assembly for CUDA

- How: Compile with specific architecture flags to avoid

- Optimize CUDA Event Timing

- Why: Events only guarantee order of operations within a single stream

- In events with multiple streams, GPU can switch between streams in between CUDA events. Could see inflated results like with task preemption

CUDA Events can add overhead

- How: Use Nsight for precise measurements of specific kernels

Or cudaEventDisableTiming to remove overhead

- Eliminate API contention and overhead

- Why: CUDA API uses mutex locks that depend on OS. Multiple CPU threads try to launch and can contend for the locks, spiking call times

- How: Best practice

- Submit all work from single CPU worker thread or CUDA Graphs for more optimization

- Gives GPU sequence of operations/dependencies